

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

OPTICROP

Smart Agricultural Production Optimization Engine

An Internship Project Report

submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Computer Science & Engineering

Submitted by

Mohan Sala

OPTICROP TECHNOLOGIES

August 2026

Declaration

I, Mohan Sala, hereby declare that the project report entitled “OptiCrop: Smart Agricultural Production Optimization Engine” submitted to the Department of Computer Science & Engineering is an original work done by me. The results embodied in this report have not been submitted to any other University or Institute for the award of any degree.

- **Date:** August 3, 2026
- **Signature of the Candidate:** Mohan Sala

Acknowledgement

I would like to express my sincere gratitude to everyone who supported and guided me throughout the course of this project. Their valuable suggestions, constructive feedback, and constant encouragement played an important role in helping me complete this work successfully.

I am also thankful to the institution for providing the necessary infrastructure, resources, and learning environment that made this project possible. Finally, I extend my heartfelt thanks to my family, friends, and peers for their continuous support and motivation during this work.

Abstract

- **Project Overview:** OptiCrop, the Smart Agricultural Production Optimization Engine, is a full-stack agricultural decision support platform combining a FastAPI backend with a React frontend to help users make data-driven farming decisions.
- **Problem:** Farmers and agronomists often lack an integrated, data-driven tool that combines crop recommendation, soil suitability analysis, and predictive analytics in one accessible interface.
- **Solution:** A versioned REST API handling authentication, datasets, preprocessing, model training, predictions, analytics, monitoring, and deployments, paired with a modern React dashboard.
- **Technology:** Built using FastAPI, SQLAlchemy, Alembic, PostgreSQL, and Pydantic on the backend; React 19, TanStack Router, TanStack Query, Vite, and Tailwind CSS on the frontend.
- **Outcome:** A modular, testable platform that supports the full machine-learning lifecycle — from dataset ingestion to model deployment — alongside a data-rich user dashboard.

Table of Contents

1. Introduction.....	6
2. Problem Statement.....	6
3. Objectives.....	6
4. Scope of the Project.....	6
5. Feasibility Study	7
6. System Requirements	7
7. Technology Stack	7
8. Repository Structure	8
9. Backend Architecture.....	9
10. Machine Learning Pipeline	9
11. Data Access & Repository Layer	10
12. Security & Access Control.....	10
13. Error Handling & Exception Architecture	11
14. Configuration	11
15. Backend Setup & Execution.....	12
16. Backend Testing	13
17. Frontend Architecture.....	13
18. Frontend Setup & Execution.....	13
19. Frontend Design Notes.....	14
20. User Workflow Walkthrough.....	14
21. Logging & Monitoring	15
22. Non-Functional Requirements.....	15
23. Development Workflow	15
24. Risk Management	16
25. Advantages	16
26. Limitations	16
27. Future Enhancements	17
28. Conclusion	17
29. References.....	17
30. Appendix A — Abbreviations.....	17
31. Appendix B — Glossary	18

1. Introduction

Agriculture increasingly depends on data-driven decision-making to improve yield, optimise resource use, and reduce risk. OptiCrop was built to bring together the scattered tools a farmer or agronomist would otherwise need — crop recommendation, soil suitability checking, predictive analytics, and monitoring — into a single, coherent platform.

The system is organised as two independent applications: a FastAPI backend that owns all application data and machine-learning operations, and a React frontend that provides the user-facing dashboard, forms, and visualisations. This separation allows the backend's versioned API to be consumed by the web client today and by other clients (mobile, third-party integrations) in the future without modification.

This report documents the architecture, technology choices, module responsibilities, configuration, setup process, and testing strategy of OptiCrop as observed and worked on during the internship.

2. Problem Statement

Decision support for agriculture is frequently fragmented across spreadsheets, static advisory documents, and disconnected tools. There is rarely a single system that ingests raw agricultural datasets, preprocesses them, trains and evaluates predictive models, and then surfaces the resulting recommendations to an end user through an accessible interface.

OptiCrop addresses this by providing a unified pipeline: datasets are uploaded and preprocessed through the API, models are trained and evaluated, and the resulting predictions — crop recommendations and soil suitability scores — are made available through a dashboard, with analytics and monitoring endpoints to track ongoing model and system health.

3. Objectives

- **Primary Objective:** Build a full-stack platform that supports the complete lifecycle of an agricultural ML workflow: data ingestion, preprocessing, training, prediction, and monitoring.
- **Secondary Objective:** Provide an accessible, data-rich dashboard so non-technical users can view recommendations, analytics, and system status without interacting with the API directly.
- **Technical Objective:** Implement a versioned, well-tested FastAPI backend with async PostgreSQL access, and a React 19 frontend built on TanStack Router and TanStack Query.
- **Quality Objective:** Maintain a domain-focused backend test suite (auth, datasets, training, predictions, monitoring, and more) and consistent frontend linting and build checks.

4. Scope of the Project

- **Current Scope:** Authentication and profile management, dataset management and preprocessing, training and evaluation workflows, prediction generation and history, analytics, monitoring, and deployment-related operations, all exposed through a versioned REST API and consumed by a React dashboard.

- **Future Scope:** Splitting documentation into dedicated backend/frontend guides, generating an API reference directly from the FastAPI routes, and producing a formal deployment guide for staging and production environments.
- **Limitations:** The project currently uses separate backend and frontend folders rather than a single monorepo workspace, and requires manual coordination of environment variables between the two applications.

5. Feasibility Study

- **Technical Feasibility:** Built entirely on mature, well-documented open-source frameworks (FastAPI, SQLAlchemy, React, Vite), making the stack highly feasible for a small team to build and maintain.
- **Economic Feasibility:** Relies on open-source libraries and optional Supabase-hosted PostgreSQL, keeping infrastructure costs low during development.
- **Operational Feasibility:** The interactive API documentation at /docs and /redoc, combined with a data-rich dashboard, lowers the learning curve for both developers and end users.

6. System Requirements

Backend

- Python 3 runtime with a virtual environment (.venv)
- PostgreSQL database (local or Supabase-hosted)
- Dependencies listed in backend/requirements.txt

Frontend

- Node.js and npm
- A modern web browser

General

- Minimum 8 GB RAM recommended for running both applications concurrently during development

7. Technology Stack

Backend

Component	Technology
Web Framework	FastAPI
ORM	SQLAlchemy
Migrations	Alembic
Validation & Settings	Pydantic v2, Pydantic Settings

Component	Technology
Backend-as-a-Service	Supabase client support
Database	PostgreSQL (async access)
Testing	Pytest, pytest-asyncio

Frontend

Component	Technology
UI Library	React 19
Routing	TanStack Router
Server State	TanStack Query
Build Tool	Vite
Styling	Tailwind CSS
Component Library	Radix UI
Forms & Validation	React Hook Form, Zod

8. Repository Structure

The codebase is split into two top-level applications, each with a clearly separated internal structure:

```

backend/
  alembic/           Database migrations and Alembic config
  app/
    api/v1/          Versioned API router and route modules
    core/             Configuration, logging, middleware, security, roles
    database/         Session and connection setup
    dependencies/     Shared dependency helpers
    ml/               ML pipelines, preprocessing, classification, regression
    models/           SQLAlchemy models
    repositories/     Data access layer
    schemas/          Pydantic request/response schemas
    services/         Business logic
    storage/          File or model storage helpers
    utils/            Shared utilities and response helpers
    tests/            Backend test suite

frontend/
  public/            Static assets
  src/
    routes/           TanStack Router route modules
    components/       Shared UI components
    hooks/            Reusable React hooks
    lib/              Client utilities and app helpers
    styles.css        Global styling

```

9. Backend Architecture

Responsibilities

The backend is the source of truth for application data and machine-learning operations. It provides user authentication and profile handling, dataset management and preprocessing endpoints, training and evaluation workflows, prediction generation and history tracking, analytics and monitoring endpoints, and deployment-related operations.

API Layout

The API is versioned under `/api/v1` and grouped by domain:

Endpoint Group	Purpose
<code>/api/v1/health</code>	Service health checks
<code>/api/v1/auth</code>	Authentication (login, registration, tokens)
<code>/api/v1/profile</code>	User profile handling
<code>/api/v1/datasets</code>	Dataset management
<code>/api/v1/preprocessing</code>	Dataset preprocessing operations
<code>/api/v1/training</code>	Model training and evaluation
<code>/api/v1/analytics</code>	Analytics endpoints
<code>/api/v1/predictions</code>	Prediction generation and history
<code>/api/v1/monitoring</code>	System and model monitoring
<code>/api/v1/deployments</code>	Deployment-related operations

The root application also exposes `/` for a basic online status response, with interactive documentation available at `/docs` (Swagger UI) and `/redoc` (ReDoc).

Startup Behaviour

The FastAPI app initialises logging during lifespan startup, registers middleware, includes the versioned router, and installs global exception handlers for application-specific exceptions, validation errors, HTTP exceptions, and unhandled server exceptions.

10. Machine Learning Pipeline

The `app/ml` package is where the platform's predictive intelligence lives. It is organised as a pipeline of discrete stages so that each concern — preprocessing, classification, and regression — can be developed, tested, and versioned independently.

Preprocessing

Raw agricultural datasets uploaded through the `/api/v1/datasets` and `/api/v1/preprocessing` endpoints are cleaned, normalised, and transformed into a format suitable for model training. This stage is responsible for handling missing values, encoding categorical soil and crop attributes, and scaling numerical features such as rainfall, temperature, and nutrient levels.

Classification

Classification models underpin the crop recommendation workflow, mapping a given combination of soil and climate features to the most suitable crop categories for a plot of land.

Regression

Regression models support soil suitability scoring and yield-related predictions, producing continuous outputs (such as a suitability score or expected yield) rather than discrete categories.

Training & Evaluation

The `/api/v1/training` endpoints trigger the pipeline end to end: preprocessed data is fed into the selected model type, the resulting model is evaluated against a held-out portion of the dataset, and the outcome is persisted for later use in the `/api/v1/predictions` endpoints.

11. Data Access & Repository Layer

The backend follows a layered architecture that keeps data access logic separate from business logic and API routing. Requests flow from the versioned API routes into services, which encapsulate business rules, and from there into the repositories layer, which is solely responsible for translating those rules into SQLAlchemy queries.

1. API route receives and validates the incoming request using a Pydantic schema
2. The route delegates to the corresponding service function
3. The service applies business rules and calls into one or more repository methods
4. The repository executes the async SQLAlchemy query against PostgreSQL
5. The result is mapped back into a Pydantic response schema and returned to the client

This separation makes it straightforward to unit test business logic independently of the database, and to swap or extend persistence details without touching route handlers.

12. Security & Access Control

Security-related concerns are centralised under `app/core`, in dedicated security and roles modules, rather than being scattered across individual route handlers.

- **Authentication:** User identity is established through the `/api/v1/auth` endpoints, with JWT access tokens signed using the `JWT_SECRET` configuration value and a configurable expiry (`ACCESS_TOKEN_EXPIRE_MINUTES`).
- **Authorization:** A dedicated roles module allows endpoints to be restricted based on user role, keeping permission checks consistent across the API surface.
- **Secrets Management:** Sensitive values — `SECRET_KEY`, `JWT_SECRET`, Supabase credentials, and the database connection string — are loaded from `backend/.env` through Pydantic Settings and are never hard-coded.

- **CORS Policy:** Allowed client origins are controlled explicitly through the `BACKEND_CORS_ORIGINS` setting, restricting which frontends may call the API.

13. Error Handling & Exception Architecture

The FastAPI application installs a set of global exception handlers during startup so that errors are surfaced to API consumers in a consistent, predictable shape rather than as raw stack traces.

Handler	Responsibility
Application-specific exceptions	Domain errors raised deliberately by services (e.g. dataset not found, model not trained)
Validation errors	Pydantic schema validation failures on incoming request bodies or query parameters
HTTP exceptions	Explicit HTTP error responses raised by route handlers
Unhandled server exceptions	A catch-all handler that prevents internal errors from leaking implementation details to the client

14. Configuration

Backend configuration is loaded from `backend/.env` through Pydantic Settings. Key environment variables include:

Variable	Purpose
<code>APP_NAME / APP_VERSION</code>	Application identity and version
<code>ENVIRONMENT / DEBUG</code>	Runtime environment and debug flag
<code>HOST / PORT</code>	Server bind address and port
<code>API_V1_PREFIX</code>	Versioned API path prefix
<code>SECRET_KEY / JWT_SECRET</code>	Cryptographic signing keys
<code>ACCESS_TOKEN_EXPIRE_MINUTES</code>	JWT access token lifetime
<code>SUPABASE_URL / ANON_KEY / SERVICE_ROLE</code>	Supabase project credentials
<code>DATABASE_URL</code>	Async PostgreSQL connection string
<code>STORAGE_BUCKET</code>	File storage bucket reference
<code>BACKEND_CORS_ORIGINS</code>	Allowed client origins
<code>UPLOAD_PATH / MODEL_PATH</code>	Local storage paths for uploads and models
<code>LOG_LEVEL</code>	Logging verbosity

Example `backend/.env`

```

APP_NAME=OptiCrop Backend
APP_VERSION=1.0.0
ENVIRONMENT=development
DEBUG=true
HOST=127.0.0.1
PORT=8000
API_V1_PREFIX=/api/v1
SECRET_KEY=replace-me
JWT_SECRET=replace-me-too
DATABASE_URL=postgresql+asyncpg://user:password@localhost:5432/opticrop
BACKEND_CORS_ORIGINS=http://localhost:3000,http://localhost:5173,http://localhost:5174
UPLOAD_PATH=tmp/uploads
MODEL_PATH=models
LOG_LEVEL=INFO

```

15. Backend Setup & Execution

Environment Setup

```

cd backend
python -m venv .env
.venv\Scripts\activate
pip install -r requirements.txt

```

Running the Backend

```

cd backend
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000

```

Useful URLs while developing: <http://127.0.0.1:8000/>, <http://127.0.0.1:8000/docs>, and <http://127.0.0.1:8000/redoc>.

Database Migrations

Alembic is configured under backend/alembic/. The common workflow is to apply migrations before running integration tests:

```

cd backend
alembic upgrade head

```

16. Backend Testing

The backend includes a domain-focused test suite under backend/tests/, covering analytics, authentication, datasets, deployments, health checks, monitoring, predictions, preprocessing, profile, and training.

```

cd backend
pytest

```

17. Frontend Architecture

Responsibilities

The frontend is the user-facing application. It provides public auth pages for login and registration, an application shell under /app, dashboards and analytics views, crop recommendation and suitability workflows, and settings and model-related pages.

Routing

The route structure is defined in frontend/src/routes/:

Route	Purpose
/	Landing page
/login	User login
/register	User registration
/app	Application shell (redirects to /app/dashboard)
/app/dashboard	Main dashboard with KPIs and quick actions
/app/analytics	Analytics views
/app/models	Model-related pages
/app/recommendation	Crop recommendation workflow
/app/suitability	Soil suitability workflow
/app/settings	Account and application settings

18. Frontend Setup & Execution

Installing Dependencies

```
cd frontend
npm install
```

Running the Frontend

```
cd frontend
npm run dev
```

Other Available Scripts

- npm run build
- npm run build:dev
- npm run preview
- npm run lint

- npm run format

19. Frontend Design Notes

The frontend uses TanStack Router for file-based routing, TanStack Query for server-state management, and Radix UI paired with Tailwind CSS for the design system. The dashboard implementation is intentionally data-rich, showing KPI cards, recent predictions, and quick actions to common workflows, giving users an immediate overview of system activity on login.

20. User Workflow Walkthrough

This section traces a typical end-to-end user journey through the application, tying the frontend routes to the backend endpoints they rely on.

1. Account Access

A new user registers through `/register`, which calls `/api/v1/auth` to create an account, and subsequently signs in through `/login`. On success, the application shell under `/app` becomes accessible and the user is redirected to `/app/dashboard`.

2. Dashboard Overview

The dashboard aggregates KPI cards, recent predictions, and quick actions, drawing on the analytics, predictions, and monitoring endpoints to present an immediate snapshot of system activity.

3. Crop Recommendation

From `/app/recommendation`, a user submits soil and climate parameters, which are routed through `/api/v1/predictions` to the trained classification model, returning a ranked list of suitable crops.

4. Soil Suitability

The `/app/suitability` route follows a similar pattern, but calls into the regression side of the machine learning pipeline to produce a continuous suitability score for a given plot.

5. Analytics & Models

The `/app/analytics` and `/app/models` routes surface historical trends and details about trained models respectively, backed by the `/api/v1/analytics` and `/api/v1/training` endpoints.

6. Settings

The `/app/settings` route allows the user to manage their profile and account preferences via the `/api/v1/profile` endpoints.

21. Logging & Monitoring

Observability is treated as a first-class concern rather than an afterthought. Logging is initialised during the FastAPI application's lifespan startup, before any request is served, and its verbosity is controlled through the `LOG_LEVEL` configuration value.

- **Application Logging:** Startup, shutdown, and request-level events are logged consistently so that issues can be traced back to a specific stage of the request lifecycle.
- **Monitoring Endpoints:** The `/api/v1/monitoring` endpoint group exposes system and model health information, allowing operators to track the status of trained models and the backend service over time.
- **Health Checks:** The `/api/v1/health` endpoint, together with the root `/` endpoint, provides a lightweight way to confirm the service is online, and is well suited to being polled by uptime monitors or container orchestration health probes.

22. Non-Functional Requirements

- **Performance:** Async database access throughout the backend (async PostgreSQL via SQLAlchemy) is intended to keep the API responsive under concurrent load, particularly during dataset preprocessing and training operations.
- **Scalability:** The versioned API and clear separation between backend and frontend allow either side to be scaled or redeployed independently as usage grows.
- **Maintainability:** A layered backend structure (routes, services, repositories) and a domain-focused test suite keep the codebase approachable as new features are added.
- **Usability:** A data-rich dashboard and interactive API documentation (`/docs`, `/redoc`) reduce the effort required for both end users and developers to understand the system.
- **Security:** Centralised authentication, role-based access control, and environment-variable-driven secrets management (see Section 12) protect sensitive agricultural and account data.

23. Development Workflow

1. Start the backend and confirm `/docs` loads
2. Apply or verify database migrations
3. Start the frontend dev server
4. Walk through auth, dashboard, prediction, analytics, and monitoring flows
5. Run the backend test suite before merging backend changes
6. Run frontend linting and production builds before merging UI changes

24. Risk Management

A number of project and technical risks were identified and considered during development, along with the mitigations reflected in the current design.

Risk	Mitigation
Coordination drift between backend and frontend environment variables	Environment variables are documented centrally and validated through Pydantic Settings at startup, failing fast on misconfiguration.
Data quality issues in uploaded agricultural datasets	A dedicated preprocessing stage cleans and validates data before it reaches training, reducing the risk of models trained on malformed input.
Unauthorised access to sensitive account or prediction data	Centralised JWT authentication and role-based access control (Section 12) restrict endpoint access by identity and role.
Silent model or service degradation in production	Dedicated monitoring and health-check endpoints (Section 21) surface issues before they affect end users.
Schema drift between the database and application models	Alembic migrations version-control every schema change, keeping the database and SQLAlchemy models in sync.

25. Advantages

- Single versioned API cleanly separates the backend from the presentation layer, allowing multiple future clients.
- Covers the full ML lifecycle end to end: ingestion, preprocessing, training, prediction, and monitoring.
- Interactive API documentation (/docs, /redoc) speeds up both development and onboarding.
- Data-rich dashboard gives non-technical users immediate visibility into recommendations and system health.

26. Limitations

- Backend and frontend are managed as separate projects rather than a unified monorepo workspace, requiring manual coordination.
- New backend environment variables must be documented manually in both the README and app/core/config.py.
- No dedicated deployment guide currently exists for staging or production environments.

27. Future Enhancements

- **Documentation Split:** Break the documentation into dedicated docs/backend.md and docs/frontend.md files.
- **Generated API Reference:** Produce an API reference generated directly from the FastAPI routes.
- **Deployment Guide:** Author a deployment guide covering local, staging, and production environments.
- **Monorepo Tooling:** Evaluate consolidating the backend and frontend into a single workspace-managed repository.

28. Conclusion

OptiCrop demonstrates how a modern, versioned FastAPI backend and a React 19 frontend can be combined to deliver a complete agricultural decision support workflow — from raw dataset upload through preprocessing, training, prediction, and monitoring, to a clear and data-rich dashboard. The project also provided practical experience in async API design, database migrations, domain-focused testing, and modern frontend routing and state management.

29. References

- FastAPI Documentation – fastapi.tiangolo.com
- SQLAlchemy Documentation – sqlalchemy.org
- Alembic Documentation – alembic.sqlalchemy.org
- Pydantic Documentation – docs.pydantic.dev
- React Documentation – react.dev
- TanStack Router & Query – tanstack.com
- Tailwind CSS Documentation – tailwindcss.com
- Radix UI Documentation – radix-ui.com
- PostgreSQL Documentation – postgresql.org/docs

30. Appendix A — Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
ORM	Object-Relational Mapping
JWT	JSON Web Token
REST	Representational State Transfer
KPI	Key Performance Indicator
ML	Machine Learning
UI / UX	User Interface / User Experience
CORS	Cross-Origin Resource Sharing

31. Appendix B — Glossary

Term	Definition
Crop Recommendation	A prediction that maps soil and climate inputs to the most suitable crops for a plot of land.
Soil Suitability	A continuous score indicating how well-suited a given plot of land is for a particular crop or use.

Term	Definition
Preprocessing	The stage where raw datasets are cleaned, normalised, and transformed before being used to train a model.
Training	The process of fitting a machine learning model to a prepared dataset.
Repository Layer	The part of the backend responsible for translating business operations into database queries.
Service Layer	The part of the backend responsible for business logic, sitting between API routes and repositories.
Lifespan Startup	The FastAPI mechanism used to run initialisation code, such as logging setup, when the application starts.
KPI Card	A dashboard element summarising a single key performance indicator, such as total predictions made.